

Lecture 19

MVU I

CS 12 Team 25.2

A.Y. 2025–2026, 2nd Sem

Department of Computer Science

College of Engineering

University of the Philippines Diliman

Sanity Check

- Attendance?
- Audible at the back?
- TV(s) on?
 - (we're working on getting the other TV fixed...)
- **Recording?**
- Important announcements?

Motivating Problem

Create a single-page web app (SPA) showing an *integer counter*

- Counter starts at 0
- + button *increments counter* by 1
- - button *decrements counter* by 1

How do we go about designing this?

Step 1: Represent State of App

What data does the app need to keep track of?

- Value of counter: `Int` with *initial value* 0

Store needed data as Model

Step 1: Represent State of App

- In Elm:

```
type alias Model = Int
```

```
init = 0 -- Starting state
```

```
-- What if we need multiple pieces of data?
```

Step 2: Identify How Users Can Affect State

How does the user interact with the web app?

User interaction

Effect on state

+ button is pressed	Increment state by 1
- button is pressed	Decrement state by 1

Step 2.1: Represent All Possibilities of Interactions

Define new `Msg` type as *tagged union* with *possibilities as variants*

- In Elm:

```
-- Variants can have any name; Msg prefix is for readability
type Msg
    = MsgIncrement    -- For when + is pressed
    | MsgDecrement    -- For when - is pressed
```

Step 2.2: Define How State Changes Based on Interaction

Define new `update: Msg -> Model -> Model`

```
update: Msg -> Model -> Model -- Recall that Model == Int
update msg model =
  case msg of
    MsgIncrement ->
      model + 1
    MsgDecrement ->
      model - 1
```


Step 3: Define UI based on state

What parts of the UI change based on the state?

- Define `view: Model -> Html Msg` in Elm:

```
import Html exposing (Html, button, div, text)
import Html.Events exposing (onClick)
```

Step 3: Define UI based on state

```
view : Model -> Html Msg
view model =
  -- Will explain parts in detail later
  div []
    -- Children
    [ button [ onClick MsgIncrement ] [ text "+" ]
    , p [] [ text (String.fromInt model) ]
    , button [ onClick MsgDecrement ] [ text "-" ]
    ]
```

Step 4: Model + View + Update + Msg + Main

```
import Browser
import Html exposing (Html, button, div, p, text)
import Html.Events exposing (onClick)
```

```
type alias Model = Int
```

```
init = 0
```

```
type Msg
    = MsgIncrement
    | MsgDecrement
```

Step 4: Model + View + Update + Msg + Main

```
view : Model -> Html Msg
view model =
  div []
    [ button [ onClick MsgIncrement ] [ text "+" ]
    , p [] [ text (String.fromInt model) ]
    , button [ onClick MsgDecrement ] [ text "-" ]
    ]
```

Step 4: Model + View + Update + Msg + Main

```
update : Msg -> Model -> Model
update msg model =
  case msg of
    MsgIncrement ->
      model + 1
    MsgDecrement ->
      model - 1
```

Step 4: Model + View + Update + Msg + Main

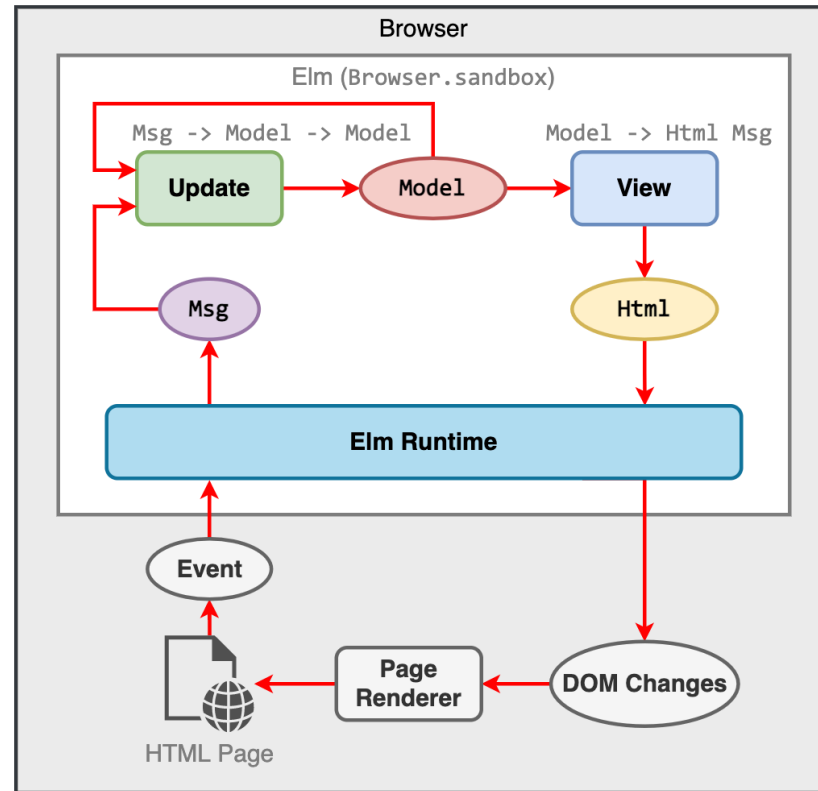
```
main =  
    Browser.sandbox  
        { init = init  
          , update = update  
          , view = view }
```

MVU Architecture

MVU: Model-View-Update

- Also called *TEA*: (The Elm Architecture)
- One way of organizing interactive programs
- **Model**: *State* of the program
- **View**: *UI* based on state
- **Update**: *State update via messages*

Basic MVU Cycle



Practice: Add a Reset Button

- Would `Model` need to change?
- Would `init` need to change?
- Would `Msg` need to change?
- Would `Update` need to change?
- Would `View` need to change?

Demo: Add a Reset Button

Task: Add a Reset Button

- Possible solution:

```
type Msg
  = MsgIncrement
  | MsgDecrement
  | MsgReset  -- New message for even when user clicks reset
button
```

Task: Add a Reset Button

```
-- New button; View must change accordingly
view : Model -> Html Msg
view model =
  div []
    [ button [ onClick MsgIncrement ] [ text "+" ]
    , p [] [ text (String.fromInt model) ]
    , button [ onClick MsgDecrement ] [ text "-" ]
    -- New button that raises MsgReset on click
    , button [ onClick MsgReset ] [ text "Reset" ]
    ]
```

Task: Add a Reset Button

```
-- New message; Update must handle it
update : Msg -> Model -> Model
update msg model =
    case msg of
        MsgIncrement -> model + 1
        MsgDecrement -> model - 1
        -- State becomes 0 when MsgReset is raised
        MsgReset -> 0
```

MVU Characteristics

- In line with FP principles:
 - **Model** and **Msg** are *immutable data*
 - **View** and **Update** are *pure functions*
- Entire program state is in **Model**
 - Easy to inspect, test, and time-travel debug
 - Easy to make state persistent
- *Unidirectional* high-level *program and data flow*

A `View` function earlier

- `Html Msg` is an HTML DOM tree:

A **View** function earlier

```
import Html exposing (Html, button, div, text)
import Html.Events exposing (onClick)
```

```
view : Model -> Html Msg
view model =
  div []
    [ button [ onClick MsgIncrement ] [ text "+" ]
    , p [] [ text (String.fromInt model) ]
    , button [ onClick MsgDecrement ] [ text "-" ]
    , button [ onClick MsgReset ] [ text "Reset" ]
    ]
```


A `View` function earlier

- *Note:* Results are only one `Html` `m` value

Div node

- `div` is a block *container* for other HTML nodes:

```
import Html exposing (div)
```

```
div : List (Attribute m) -> List (Html m) -> Html m
```

Div node

```
-- ...  
div [] -- No attributes  
  -- Four children: List (Html m)  
  [ button [ onClick MsgIncrement ] [ text "+" ]  
    , p [] [ text (String.fromInt model) ]  
    , button [ onClick MsgDecrement ] [ text "-" ]  
    , button [ onClick MsgReset ] [ text "Reset" ]  
  ]
```

P node

- p represents a *paragraph* block containing text:

```
import Html exposing (p)
```

```
-- Same signature as div
```

```
p : List (Attribute m) -> List (Html m) -> Html m
```

P node

```
-- ...  
div []  
  [ ...  
    -- No attributes, one child node  
    , p [] [ text (String.fromInt model) ]  
    , ...  
    , ...  
  ]
```

Text node

- text represents textual content:

```
import Html exposing (text)
```

```
-- Not the same as the previous ones  
text : String -> Html m
```

Text node

```
-- ...  
div []  
  [ ...  
    -- Text content is string form of state  
    , p [] [ text (String.fromInt model) ]  
    , ...  
    , ...  
  ]
```

Button node

- `button` represents an *HTML form button*:

```
import Html exposing (button)
```

```
button : List (Attribute m) -> List (Html m) -> Html m
```


Button node

```
-- ...  
  div []  
    -- One attribute, one child node  
    [ button [ onClick MsgIncrement ] [ text "+" ]  
  
      , ...  
  
    -- One attribute, one child node  
    , button [ onClick MsgDecrement ] [ text "-" ]  
  
    -- One attribute, one child node
```

```
, button [ onClick MsgReset ] [ text "Reset" ]  
]
```

OnClick attribute

- `onClick` defines what `Msg` is raised when the node raises a click event:

```
import Html.Events exposing (onClick)
```

```
-- Self-check: Using types, prove why `onClick` should be  
--               part of the first list of `button`  
onClick : m -> Attribute m
```

OnClick attribute

```
-- ...  
div []  
  -- Raise `MsgIncrement` when clicked  
  [ button [ onClick MsgIncrement ] [ text "+" ]  
  
    , ...  
  
    -- Raise `MsgDecrement` when clicked  
    , button [ onClick MsgDecrement ] [ text "-" ]  
  
    -- Raise `MsgReset` when clicked
```

```
, button [ onClick MsgReset ] [ text "Reset" ]  
]
```

More info

- HTML nodes ([link](#))
 - h1 to h6, input
- HTML events ([link](#))
- CSS styling + HTML attributes ([link](#))

Self-check

- Task 1: Add a button that negates the current counter value
- Task 2: Add a button that doubles the current counter value
- Task 3 (challenge): Add a button that adds the number in an input textbox to the counter value (hint: store this in the Model as well + use the `onInput` event)